

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN
THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 2

BÁO CÁO MÔN HỌC

NGÔN NGỮ LẬP TRÌNH C++

Nghiên cứu và Ứng dụng Thuật toán
Tìm kiếm Nhị phân (Binary Search)
trên mảng đã sắp xếp

Sinh viên thực hiện:

1. Trịnh Bùi Đức Cảnh - MSSV: N25DECE004

2. Lý Anh Khoa - MSSV: N25DECE018

Lớp: E25CQCE01-N

Thành phố Hồ Chí Minh, Ngày 27 tháng 05 năm 2026

Mục lục

1	Lời mở đầu	2
1.1	Giới thiệu về bài báo cáo	2
1.2	Mục đích của thuật toán Tìm kiếm nhị phân (tổng quát)	2
2	Giới thiệu	2
2.1	Giới thiệu thuật toán	2
2.2	Ứng dụng của thuật toán trong ngành CNTT và đời sống	2
2.3	Độ phức tạp thuật toán và chứng minh độ phức tạp	3
3	Lập trình	3
3.1	Các hàm có sẵn của Tìm Kiếm Nhị Phân trên C++ (Thư viện STL)	3
3.2	Mã nguồn lập trình tự thân	4
3.2.1	Tìm kiếm nhị phân cơ bản (Sử dụng vòng lặp while)	4
3.2.2	Tìm kiếm nhị phân cơ bản (Sử dụng đệ quy)	4
3.2.3	Biến thể 1: Tìm vị trí xuất hiện ĐẦU TIÊN của phần tử X (first_pos)	4
3.2.4	Biến thể 2: Tìm vị trí xuất hiện CUỐI CÙNG của phần tử X (last_pos)	5
3.3	Những lưu ý khi lập trình tìm kiếm nhị phân	5
4	Tổng kết	6

1 Lời mở đầu

1.1 Giới thiệu về bài báo cáo

Bài báo cáo này được thực hiện trong khuôn khổ môn học nhằm nghiên cứu, phân tích sâu và hệ thống hóa kiến thức về thuật toán Tìm kiếm Nhị phân (Binary Search) trong ngôn ngữ lập trình C++. Nội dung bài báo cáo tập trung làm rõ từ nền tảng lý thuyết toán học, chứng minh độ phức tạp thuật toán cho đến việc khảo sát các hàm hỗ trợ sẵn trong thư viện chuẩn C++ (STL). Đồng thời, bài báo cáo cũng trình bày các kỹ thuật tự triển khai mã nguồn cho các biến thể nâng cao nhằm tối ưu hóa hiệu năng xử lý dữ liệu trong thực tế lập trình thi đấu và phát triển phần mềm.

1.2 Mục đích của thuật toán Tìm kiếm nhị phân (tổng quát)

Mục đích tổng quát của thuật toán Tìm kiếm Nhị phân là tối ưu hóa tốc độ và hiệu suất xác định vị trí của một phần tử cụ thể (hoặc kiểm tra sự tồn tại của nó) trong một không gian dữ liệu kích thước lớn đã được sắp xếp theo một thứ tự nhất định. Bằng việc áp dụng tư duy “chia để trị”, thuật toán nhằm liên tục loại bỏ một nửa số lượng phần tử không khả thi sau mỗi lần thực hiện phép so sánh. Mục tiêu tối thượng của phương pháp này là giảm thiểu số bước xử lý từ tuyến tính $\mathcal{O}(n)$ xuống mức logarit $\mathcal{O}(\log n)$, giúp hệ thống máy tính truy xuất thông tin một cách nhanh chóng, tiết kiệm tối đa tài nguyên thời gian và bộ nhớ.

2 Giới thiệu

2.1 Giới thiệu thuật toán

Tìm kiếm nhị phân (Binary Search) là một thuật toán tìm kiếm được sử dụng để xác định vị trí của một phần tử cụ thể (*target*) trong một mảng đã được sắp xếp. Thuật toán hoạt động theo nguyên lý **Chia để trị (Divide and Conquer)** thông qua các bước logic sau:

1. So sánh phần tử cần tìm với phần tử nằm ở chính giữa mảng (*mid*).
2. Nếu trùng khớp, quá trình tìm kiếm thành công và trả về kết quả ngay lập tức.
3. Nếu phần tử cần tìm nhỏ hơn phần tử giữa, thuật toán tiếp tục lặp lại quá trình tìm kiếm ở nửa bên trái của mảng.
4. Nếu phần tử cần tìm lớn hơn phần tử giữa, quá trình tìm kiếm diễn ra ở nửa bên phải.
5. Quá trình này lặp đi lặp lại cho đến khi tìm thấy mục tiêu hoặc không gian tìm kiếm cạn kiệt (*left > right*).

2.2 Ứng dụng của thuật toán trong ngành CNTT và đời sống

- Trong Khoa học Máy tính & CNTT:

- *Truy vấn cơ sở dữ liệu (Database Indexing)*: Các hệ quản trị CSDL như MySQL, PostgreSQL sử dụng cấu trúc cây B-Tree (một dạng tổng quát của tìm kiếm nhị phân) để định chỉ mục và truy xuất dữ liệu bản ghi với tốc độ cực cao.

- *Gỡ lỗi hệ thống (Debugging)*: Công cụ lệnh `git bisect` sử dụng tìm kiếm nhị phân trên lịch sử các commit để tìm ra chính xác đoạn mã nguồn nào bắt đầu gây ra lỗi cho toàn bộ dự án.
- **Trong đời sống thực tế:**
 - *Tra từ điển / Danh bạ*: Khi tra cứu từ trong từ điển truyền thống, ta thường lật ngẫu nhiên ở giữa quyển sách, xác định chữ cái cần tìm nằm ở nửa trước hay nửa sau để tiếp tục phân đôi phạm vi tìm kiếm.
 - *Trò chơi đoán số*: Chiến thuật tối ưu nhất để tìm ra một số bí mật nằm trong khoảng từ 1 đến 100 là luôn luôn đoán số ở chính giữa khoảng khả thi hiện tại.

2.3 Độ phức tạp thuật toán và chứng minh độ phức tạp

- **Độ phức tạp thời gian (Time Complexity)**: Trường hợp tốt nhất là $\mathcal{O}(1)$ (khi phần tử cần tìm nằm ngay giữa mảng ở lần chia đầu tiên). Trường hợp trung bình và tệ nhất là $\mathcal{O}(\log n)$.
- **Độ phức tạp không gian (Space Complexity)**: Đạt $\mathcal{O}(1)$ đối với phương pháp triển khai bằng vòng lặp tuần hoàn và $\mathcal{O}(\log n)$ đối với phương pháp đệ quy do ảnh hưởng của Stack frame.

Chứng minh toán học cho trường hợp tệ nhất:

Giả sử kích thước của mảng dữ liệu ban đầu là N . Sau mỗi một vòng lặp chia đôi, kích thước không gian tìm kiếm còn lại tương ứng là:

- Vòng lặp thứ 1: $\frac{N}{2^1}$
- Vòng lặp thứ 2: $\frac{N}{2^2}$
- Vòng lặp thứ k : $\frac{N}{2^k}$

Thuật toán tìm kiếm nhị phân sẽ dừng lại khi không gian tìm kiếm bị thu hẹp hoàn toàn về mức tối thiểu, tức là còn lại 1 phần tử duy nhất. Khi đó, ta có phương trình toán học:

$$\frac{N}{2^k} = 1 \implies N = 2^k \implies k = \log_2(N)$$

Do đó, số bước thực hiện so sánh tối đa trong trường hợp xấu nhất là $\log_2(N)$, chứng minh độ phức tạp thời gian của thuật toán đạt mức $\mathcal{O}(\log n)$.

3 Lập trình

3.1 Các hàm có sẵn của Tìm Kiếm Nhị Phân trên C++ (Thư viện STL)

Trong thư viện tiêu chuẩn `<algorithm>` của C++, ngôn ngữ đã tối ưu hóa sẵn các hàm tìm kiếm nhị phân hiệu năng cao:

- `std::binary_search(a, a + n, x)`: Trả về kiểu dữ liệu `bool` (`true` nếu mảng tồn tại giá trị x , ngược lại trả về `false`).
- `std::lower_bound(a, a + n, x)`: Trả về một con trỏ (*iterator*) trỏ đến phần tử **đầu tiên** có giá trị lớn hơn hoặc bằng x ($\geq x$).
- `std::upper_bound(a, a + n, x)`: Trả về một con trỏ (*iterator*) trỏ đến phần tử **đầu tiên** có giá trị hoàn toàn lớn hơn x ($> x$).

3.2 Mã nguồn lập trình tự thân

Dưới đây là các kỹ thuật tự triển khai mã nguồn Tìm kiếm nhị phân, đặc biệt hữu ích khi cần tùy biến các điều kiện logic phức tạp trong lập trình thi đấu.

3.2.1 Tìm kiếm nhị phân cơ bản (Sử dụng vòng lặp while)

```
1 bool bs(int a[], int n, int x) {
2     int l = 0, r = n - 1;
3     while (l <= r) {
4         int m = (l + r) / 2; // Tìm chỉ số o giữa
5         if (a[m] == x)
6             return true;
7         // Thang dung giữa nhỏ hơn x, thì phải tìm ở bên phải
8         else if (a[m] < x) {
9             l = m + 1;
10        }
11        // Phải tìm ở bên trái
12        else {
13            r = m - 1;
14        }
15    }
16    return false;
17 }
```

3.2.2 Tìm kiếm nhị phân cơ bản (Sử dụng đệ quy)

```
1 bool binary_search_recursive(int a[], int l, int r, int x) {
2     if (l > r)
3         return false;
4
5     int m = (l + r) / 2;
6     if (a[m] == x)
7         return true;
8     else if (a[m] < x)
9         return binary_search_recursive(a, m + 1, r, x); // Tìm nửa
10    phải
11    else
12        return binary_search_recursive(a, l, m - 1, x); // Tìm nửa
13    trái
14 }
```

3.2.3 Biến thể 1: Tìm vị trí xuất hiện ĐẦU TIÊN của phần tử X (first_pos)

```
1 int first_pos(int a[], int n, int x) {
2     int l = 0, r = n - 1;
3     int res = -1;
4     while (l <= r) {
5         int m = (l + r) / 2;
6         if (a[m] == x) {
7             res = m; // Ghi nhận đáp án tạm thời
8             r = m - 1; // Tiếp tục tìm thêm ở bên trái
9         }
10    }
11    return res;
12 }
```

```

9         }
10        else if (a[m] < x) {
11            l = m + 1;
12        }
13        else {
14            r = m - 1;
15        }
16    }
17    return res;
18 }

```

3.2.4 Biến thể 2: Tìm vị trí xuất hiện CUỐI CÙNG của phần tử X (last_pos)

```

1 int last_pos(int a[], int n, int x) {
2     int l = 0, r = n - 1;
3     int res = -1;
4     while (l <= r) {
5         int m = (l + r) / 2;
6         if (a[m] == x) {
7             res = m;           // Ghi nhận đáp án tạm thời
8             l = m + 1;         // Tiếp tục tìm thêm ở bên phải
9         }
10        else if (a[m] < x) {
11            l = m + 1;
12        }
13        else {
14            r = m - 1;
15        }
16    }
17    return res;
18 }

```

3.3 Những lưu ý khi lập trình tìm kiếm nhị phân

1. **Tính chất tiên quyết:** Các phần tử trong mảng hoặc vector **bắt buộc** phải được sắp xếp đơn điệu (tăng dần hoặc giảm dần) trước khi áp dụng thuật toán. Nếu mảng chưa sắp xếp, mọi kết quả trả về từ Binary Search đều hoàn toàn sai lệch.
2. **Lỗi tràn số nguyên (Integer Overflow):** Khi xử lý tập dữ liệu có kích thước cực lớn, công thức tính phần tử giữa thông thường $m = (l + r) / 2$ có thể làm vượt quá giới hạn của kiểu dữ liệu `int` do phép cộng $(l + r)$. Khắc phục triệt để bằng công thức an toàn:

$$m = l + (r - l) / 2$$

3. **Tư duy cập nhật cận trong các bài toán biến thể:** Trong các hàm tìm vị trí biên như `first_pos` hay `last_pos`, khi điều kiện $a[m] == x$ thỏa mãn, ta không được phép dừng và ngắt hàm ngay, mà phải lưu lại chỉ số tiềm năng vào biến `res` rồi liên tục ép ranh giới dữ liệu dịch chuyển sang trái hoặc sang phải để quét sạch không gian tìm kiếm.

4 Tổng kết

Kiến thức đúc kết:

Bài báo cáo đã làm rõ bản chất lý thuyết, toán học và kỹ thuật lập trình của thuật toán Tìm kiếm Nhị phân. Tư duy chia để trị trong thuật toán này là nền tảng cốt lõi cho nhiều cấu trúc dữ liệu nâng cao sau này như Cây tìm kiếm nhị phân (BST), Cây đoạn thẳng (Segment Tree) cũng như giải quyết bài toán tìm kiếm kiếm kết quả tối ưu (Binary Search on Answer).

Thuận lợi:

- Thư viện C++ STL hỗ trợ cực kỳ mạnh mẽ thông qua các hàm có sẵn (`binary_search`, `lower_bound`, `upper_bound`), giúp tăng tốc độ viết mã nguồn trong thực tế.
- Thuật toán có tính logic cao, mô phỏng trực quan tốt dựa trên thực tế cuộc sống. Việc nắm giữ biến thể vị trí biên giúp giải quyết trọn vẹn bài toán tính toán tần suất xuất hiện dữ liệu trùng lặp.

Khó khăn:

- Quá trình tự mã hóa các biến thể nâng cao đòi hỏi khả năng tư duy biên (*boundary logic*) cực kỳ tỉ mỉ, lập trình viên rất dễ mắc sai lầm gây kẹt vòng lặp vô hạn nếu phân chia khoảng `l` và `r` không chuẩn xác.
- Việc kiểm soát lỗi tràn bộ nhớ hoặc xử lý ranh giới mảng đòi hỏi sự tích lũy kinh nghiệm thực hành liên tục qua các bài toán lập trình thực tế.